HackYeah 2026 · Superteam Poland Track · Trustless finance

Get paid for work without trusting a platform with the money.

A milestone escrow that lives entirely on Solana. The client locks the full amount in a vault only the program can move. The freelancer sees it before starting. Silence has a deadline.

Program	Program size	Stack
`GPmJtpLs7u7BCfUHjRyaffjPSKuJjEjLY1qztzgtVgpC`	`6 instructions · 1 account · 126 bytes`	`Anchor 0.30.1 · SPL Token · vanilla JS app`

THE PROBLEM

Two people who have never met need a third one to hold the money.

And that third party is not a neutral pipe. It is a business with its own incentives, and every property that makes it useful is a property it can also abuse.

01

Custody

The platform holds the balance. Your money is a row in its database, withdrawable when it decides — and frozen when it decides otherwise.

02

Discretion

Disputes are settled by an employee reading messages, with no rule published in advance and no appeal that is not the same employee again.

03

Permission

Being paid requires an account, a country, and a reputation score. The work can be finished and still not be payable.

The fee is the visible cost. The invisible one is that neither side can verify the other's money exists before the work starts — which is exactly why the platform gets to charge it.

THE MECHANISM

Four instructions replace the middleman.

No custody, no dispute desk, no account. The rules are the program, and the program is 126 bytes of readable state.

CREATE

Client locks the whole job

The amount moves from the client's token account into a vault PDA. The escrow account is derived from `[client, mint, deadline]`, so the address is predictable before it exists.

→

ACCEPT

Freelancer sees the money

Accepting binds their key to the escrow. They are now working against a balance they can read on-chain, not a promise.

→

APPROVE

Each milestone releases

One approval moves `amount / milestones`. Neither side can take the whole pot early, and a remainder is rejected at creation so nothing can be stranded.

→

CLAIM

Silence has a terminus

If the client never reviews, the freelancer claims the balance after the deadline. The platform's only irreplaceable job — deciding what happens when someone goes quiet — is now a timestamp.

Why this is not just "escrow on a chain". The deadline branch is what makes it a *substitute* rather than a tool. Without it, a client who stops answering keeps the money forever and you still need a platform to complain to.

WHAT THE PROGRAM ENFORCES

Every promise is a constraint, not a policy.

PROPERTY	HOW IT IS ENFORCED
The client cannot take it back	After acceptance the vault is owned by the escrow PDA. `cancel` requires `status == Open`, so once work has started the refund path is closed to the client.
The freelancer cannot take it early	`claim_unreviewed` requires the on-chain clock to have passed `deadline`. The clock is the chain's, not the server's.
Only the right person acts	Each instruction declares its signer; `has_one` binds the escrow to the client, and the payout address is checked against `escrow.mint`.
No funds are stranded	Creation rejects an amount that does not divide evenly by the milestone count, so the vault can always be emptied by approvals or by a claim.
Two jobs cannot collide	The PDA includes the client, the mint and the deadline, so a second job with identical terms cannot silently overwrite the first.

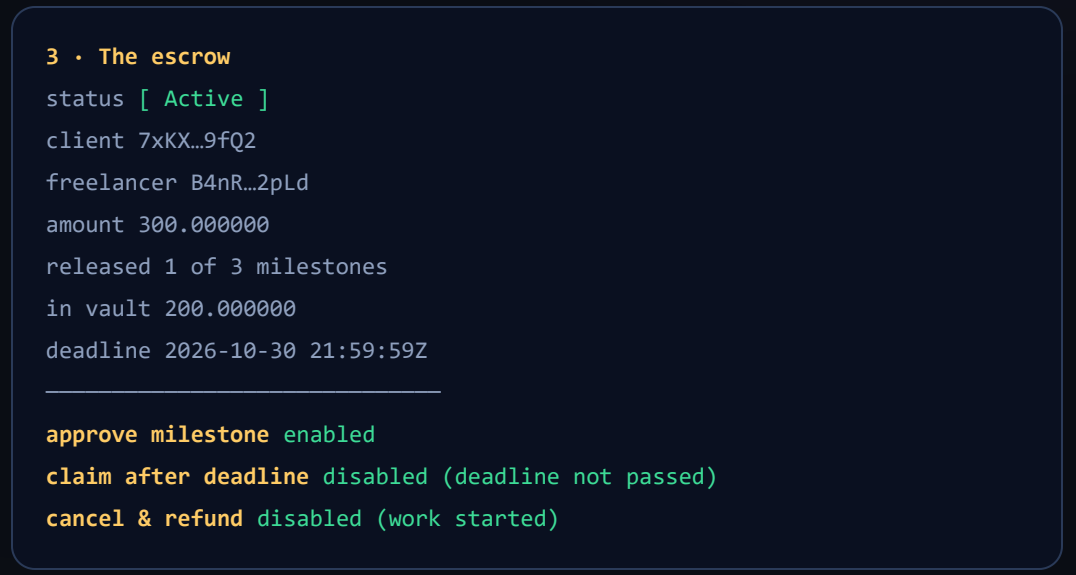
Deliberately absent: an admin key, an upgrade-authority escape hatch in the instruction set, and any instruction that can move the vault without either a milestone approval or a passed deadline.

THE PRODUCT

It is a page you can open, not a diagram of one.

One file. No build step, no bundler, no npm install. Connect a browser wallet, mint demo money, post a job, accept it from a second wallet, release it milestone by milestone, and watch the escrow state change on the explorer.

- **Decoded, not raw.** The panel renders the account — client, freelancer, mint, released milestones, vault balance, deadline — instead of an address list.
- **Role-aware buttons.** A button that would fail is disabled; the program is still the authority, this is only courtesy.
- **Honest when it cannot work.** If the program is not at the configured address the page says so in a banner, rather than letting six buttons fail one at a time.



The escrow panel, as the app renders it.

WHAT SHIPS

Small enough to audit in one sitting.

PROGRAM

Anchor 0.30.1

6 instructions, 1 account, 126 bytes, two PDAs. No admin instruction, no upgrade escape hatch, no instruction that moves the vault outside an approval or a passed deadline.

TESTS

18 scenarios

Happy path, both refund paths, every rejection, double-approval, claim before and after the deadline, and the arithmetic that keeps the vault from stranding a remainder.

APP

Single file, no build

Encodes the six instructions itself, so the page never loads a second copy of `@solana/web3.js` and never produces a phantom `instanceof PublicKey` failure.

TOOLING

Three checkers, all exit non-zero

`api-crosscheck` for source/IDL/test drift, `app-check` for discriminators and account size, `app-keycheck` which executes the shipped encoders and compares every account slot and flag to the IDL.

READ THE WHOLE THING

README, including the walls

The build instructions, the Playground failure that reads like a program bug and is not, and an explicit list of what has *not* been proven. The document that says what is missing is the one worth trusting.

VERIFICATION STATUS

What we know, and what we have not proven yet.

CLAIM	STATUS	BASIS
The program compiles and produces a real IDL	PROVEN	Clean build in Solana Playground; the IDL in the repo is its output.
The app's six instructions match the program	PROVEN	Every discriminator recomputed and every account slot compared to the IDL at runtime.
The page runs in a browser	PROVEN	Loaded, module executed, network check ran, zero console errors.
The 18 tests pass	NOT RUN	The suite starts and sends a real transaction in Playground, then stops on an exhausted devnet faucet. No payer, no test.
A transaction succeeds on chain	NOT YET	Nothing is deployed. This is the gap, and it is one `anchor deploy` away rather than an unknown.

Judges see a hundred decks claiming "fully tested, live on devnet". The repo is what they can check, so this deck only claims what the repo supports. The three rows marked proven are reproducible with two commands and no toolchain.

WHO IT IS FOR

The first job between strangers.

Not the agency with a procurement department. The person taking a first job from someone in another country, where the only thing either side knows about the other is that they are on the internet.

That is the transaction where trust has to be manufactured, and it is the one platforms charge the most for — because they are selling the existence of the money, not the transfer of it.

THE ASYMMETRY WE REMOVE

The freelancer starts work against a promise. The client pays against a hope. Both are buying the same thing: proof the other side will not vanish.

A vault anyone can read, with a rule that fires without asking, is that proof — and it does not need either party to be trusted, or even to have a bank account.

Distribution

Ships as a link. No install, no account, no KYC — the wallet is the identity, which is also the onboarding cost.

Extensibility

A second party (an arbiter) and a dispute branch are additive instructions, not a rewrite. The custody core stays untouched.

Cost

Two transaction fees and rent that is refunded on close. The platform's percentage is the thing being deleted.

WHAT HAPPENS NEXT

From a demo to a thing two strangers use.

NOW

Deploy + first live transaction

One `anchor deploy` turns the last "not yet" on slide 7 into a proven row. Everything else is already built.

NEXT

Two-wallet walkthrough

A recorded flow with a client wallet and a freelancer wallet, including the deadline branch, which no screenshot can demonstrate.

THEN

Optional arbitration

An arbiter key that can only split — never withdraw — for the disputes that a timestamp cannot settle. Opt-in, so the default stays trustless.

Business model, stated plainly. A protocol fee below 1% on release, or nothing at all with a paid dashboard for teams — both are plans, not results. The durable position is not the fee; it is being the contract other tools build on, because a wallet is cheaper to acquire than a marketplace is to build.

TRY IT

The money is the agreement.

Clone the repo, open `app/index.html` on any static server, and press the buttons. Two commands reproduce every proven row on slide 7.

```
# 1. check the client against the program
node tools/app-check.js
node tools/app-keycheck.js

# 2. run the product
python -m http.server 8080
→ http://localhost:8080/app/index.html
```

Program
GPmJtpLs7u7BCfUHjRyaffjPSKuJjEjLY1qztzgtVgpC

Track
HackYeah 2026 · Trustless finance on Solana

Repo
github.com/godd-ctrl/hackyeah-escrow